

CE 310

Python Tutorial — Sensitivity Analysis

Week 12 · Session 2 · Hands-on Walkthrough

University of Arizona · Dr. Wooyoung Jung · Fall 2026

Today's Plan — Four Inputs, One Cost

No new libraries. Everything today is arithmetic, one `for` loop, and one matplotlib call, built on the price model you fitted in Week 11.

By the end of this walkthrough you will know how to:

1. Read sensitivity straight off a fitted coefficient
2. Check it numerically with a **finite perturbation**
3. Turn a model's residual spread into a **plausible range** for an input
4. Rank four inputs with a **tornado diagram**
5. Scale a single job's exposure to a whole **awarded programme**

File needed: `CE310_BidItems_2024_2026.csv` — the same file as Week 11.

The Question Behind the Code

Week 11 asked whether a difference was real. This week asks something an estimator asks before signing: **if the numbers going in are uncertain, which one decides the answer coming out?**

💡 An input matters when the answer *responds* to it **and** you are *uncertain* about it. Either one alone is not enough. A steep response you know exactly moves nothing; a wide range on something the answer ignores moves nothing either.

That product — response × uncertainty — is what a tornado diagram ranks, and it is why today's winner is not the input you would guess.

Step 1 — Setup

 Same file and same model as Week 11 — refit it here so the coefficients are in memory.

```
import pandas as pd
import numpy as np
import statsmodels.formula.api as smf
import matplotlib.pyplot as plt

items = pd.read_csv('CE310_BidItems_2024_2026.csv')
exc = items[items['ItemDescription'] == 'EXCAV (ROADWAY)'].copy()

exc['LOW'] = (exc['LowBidder'] == 'Yes').astype(int)
exc['logq'] = np.log10(exc['Quantity'])
exc['logp'] = np.log10(exc['UnitPrice_USD'])

m2 = smf.ols('logp ~ logq + LOW', data=exc).fit()
```

Step 2 — The Coefficient IS the Sensitivity

 For a fitted model there is nothing to re-run. The coefficient already answers "how much does the output move".

```
Q0 = 2000                                # design quantity, cubic yards
logP = (m2.params['Intercept']
        + m2.params['logq'] * np.log10(Q0)
        + m2.params['LOW'] * 1)
P0 = 10 ** logP                          # the model predicts log10 price

elasticity = m2.params['logq']
print(f'price at {Q0:}, CY = ${P0:.4f}/CY')
print(f'elasticity          = {elasticity:.4f}')
```

Expected: \$21.2636 per CY · elasticity ≈ -0.1660

Step 2 (cont.) — Reading a Log–Log Slope

On a log–log fit the slope is an **elasticity**: percent change in price per percent change in quantity. To get dollars per CY, convert at the design point.

```
dP_dQ = elasticity * np.log(10) * P0 / Q0
print(f'dP/dQ = ${dP_dQ:.6f} per CY, per additional CY')
```

⚠ `np.log` is the natural log and `np.log10` is base ten. The conversion needs `np.log(10)` — the natural log *of* ten. Mixing the two is the most common error in this step.

Expected: −\$0.004063 per CY, per additional CY

Step 3 — The Finite Check

The coefficient is exact only for an infinitesimal change. Estimators work with real ones, so write the prediction as a function and move quantity 15%.

```
def price(q, low=1):  
    p = m2.params  
    return 10 ** (p['Intercept'] + p['logq'] * np.log10(q) + p['LOW'] * low)  
  
pct = 100 * (price(Q0 * 1.15) / P0 - 1)  
print(f'+15% quantity -> {pct:.4f}% on unit price')
```

Expected: $\approx -2.29\%$, against $15 \times -0.166 = -2.49\%$ from the elasticity. Close, not equal — and the gap is the curvature the linear reading ignores.

Step 4 — The Cost Model

Price is not cost. Three more inputs stand between the model's output and the number on a bid form.

$$\text{item cost} = Q \times P \times (1 + \text{overhead}) \times (1 + \text{escalation})$$

```
def cost(q, p, oh, esc):  
    return q * p * (1 + oh) * (1 + esc)  
  
BASE = dict(q=Q0, p=P0, oh=0.13, esc=0.05)  
base_cost = cost(**BASE)  
print(f'base cost = ${base_cost:,.0f}')
```

Expected: \$50,459 — hold on to it; every swing is read against this number.

Step 5 — Where the Ranges Come From

A swing is only as honest as the range behind it. Three come from practice; one comes from the model itself.

```
RMSE = np.sqrt(m2.mse_resid)      # residual spread, in log10 units
print(f'RMSE = {RMSE:.4f} -> price band '
      f'x{10**RMSE:.2f} / x{10**-RMSE:.2f}')
```

💡 `mse_resid` is the mean squared residual, so its square root is the typical miss. In $\log_{10}$ units a band of ± 1 RMSE becomes a *multiplier* on price, not an amount — which is why it is applied as `P0 * 10**RMSE`.

Expected: $\text{RMSE} \approx 0.2418 \rightarrow$ prices from $\times 0.57$ to $\times 1.75$ of the fitted value

Step 6 — Computing the Swings

Store each input's range in a dictionary, then loop. One entry per input, one swing per entry.

```
RANGES = {
    'Unit price': ('p', P0 * 10**-RMSE, P0 * 10**RMSE),
    'Quantity': ('q', Q0 * 0.85, Q0 * 1.15),
    'Overhead': ('oh', 0.08, 0.18),
    'Escalation': ('esc', 0.02, 0.08),
}

swings = {}
for label, (key, lo, hi) in RANGES.items():
    a = dict(BASE); a[key] = lo
    b = dict(BASE); b[key] = hi
    swings[label] = abs(cost(**b) - cost(**a))
```

Step 6 (cont.) — What Came Out

`dict(BASE)` copies the base case so each pass moves **one** input and leaves the other three alone. That is what the "one at a time" in OAT means.

Input	Swing	% of base
Unit price	\$59,144	117.2%
Quantity	\$15,138	30.0%
Overhead	\$4,465	8.8%
Escalation	\$2,883	5.7%

⚠ The top swing is larger than the base cost itself. That is not an error — a ± 1 RMSE band spans $\times 0.57$ to $\times 1.75$, and the distance between those two is more than one.

Step 7 — The Tornado Plot

```
order = sorted(swings, key=swings.get)      # ascending
fig, ax = plt.subplots(figsize=(9, 3.6))
ax.barh(order, [swings[k] for k in order], color='#0C234B', alpha=0.85)
ax.set_xlabel('swing in item cost, dollars')
ax.set_title(f'Tornado — {Q0:,.} CY excavation, base ${base_cost:,.0f}')
for i, k in enumerate(order):
    ax.text(swings[k], i, f' ${swings[k]:,.0f}', va='center')
ax.margins(x=0.18)
plt.show()
```

`barh` draws the first item at the **bottom**, so sorting ascending puts the longest bar on top. Sorted is what makes it a tornado rather than a bar chart.

Step 8 — Scaling to the Programme

One job is not what an agency budgets. Aggregating is two lines — and one trap.

```
won = exc[exc['LowBidder'] == 'Yes']  
prog_qty = won['Quantity'].sum()  
prog_cost = (won['Quantity'] * won['UnitPrice_USD']).sum()
```

⚠ Every project appears in this file **once per bidder**. Summing `Quantity` over all 4,369 rows returns 110,801,019 CY — 6.7× the awarded total. Note it is 6.7× and not the 4.9 bids per project you would expect: large jobs draw more bidders, so they are over-counted hardest. Filter first.

Expected: 555 awarded projects · 16,486,177 CY · \$203,833,501

Quick Check — Before You Start the In-Class Exercise

Reproduce these, then open `Wk12.03_In_Class_Exercise.ipynb`

Quantity	Expected
unit price at 2,000 CY	\$21.2636
elasticity (logq)	−0.1660
model RMSE	0.2418 log ₁₀ units
base item cost	\$50,459
top swing / bottom swing	\$59,144 / \$2,883
awarded programme quantity	16,486,177 CY

Question: unit price tops the tornado. Cost is **linear** in all four inputs
— none has a steeper response than any other. So what put it on top, and what would it take to knock it off?

Quick Reference — Sensitivity

Task	Code
Analytical sensitivity	<code>m2.params['logq']</code>
Residual spread	<code>np.sqrt(m2.mse_resid)</code>

Task	Code
Undo a $\log_{10}$ fit	<code>10 ** logP</code>
Log-slope to \$/unit	<code>b * np.log(10) * P / Q</code>

Quick Reference — Swings and Aggregation

Task	Code
Copy the base case	<code>a = dict(BASE)</code>
One input's swing	<code>abs(cost(**hi) - cost(**lo))</code>

Task	Code
Sort for a tornado	<code>sorted(swings, key=swings.get)</code>
Awarded work only	<code>df[df['LowBidder'] == 'Yes']</code>

Before You Start

- [] `CE310_BidItems_2024_2026.csv` uploaded to your Colab session
- [] Every expected value above reproduced
- [] `np.log(10)` used in the elasticity conversion, not `np.log10`
- [] Programme totals taken from winning bids only
- [] You can say in one sentence why unit price tops the tornado

Then open `Wk12.03_In_Class_Exercise.ipynb` and begin Section A.

Before You Submit

⚠ Following this submission format exactly is essential — with a class this size, grading depends on it. Submissions that don't comply will be penalized.

1. **Run all cells first.** In Colab: Runtime → Run all. Wait until every cell shows a checkmark — no ⏳ spinners remaining.
2. **Download as .ipynb.** File → Download → Download .ipynb. Do not submit PDF or .py.
3. **Do not edit the print lines.** Lines like `print(f"ANSWER_A1_base_price = {ANSWER_A1_base_price}")` must remain exactly as written — changing or deleting them means that part of your answer will not be counted.
4. **File name —** `CE310_W12_<NetID>.ipynb`, e.g., `CE310_W12_abc123.ipynb`.
5. **Upload** the `.ipynb` file to D2L Dropbox by **9:00 AM next Wednesday**.